Wissam Merhi IMEC application exercise

11/11/2026

Optimizing Token Usage in an AI Agent

Note: As this project is related to information retrieval with AI, I will be using **RAG** and my answers are based on that.

A. Identify optimization techniques

1. Top-k filtering

How it works: Setting the top-k to a lower number allows retrieving a smaller number of relevant chunks after the user asks a question. Basically, the query is embedded into a vector, and each chunk is compared with a similarity search (ex: cosine similarity) and based on that they are ranked.

Tokens reduced: input

Expected benefits: Reduced the input length by a large margin as we are just retrieving the most relevant chunks instead of a larger number of chunks. It is also said that it decreases latency and time to first token.
Possible disadvantages or risks: Loss of accuracy and useful information that are present in a wider context. This could lead to incomplete answers.

2. Conversation history compaction (through summarization)

How it works: The conversation history is usually concatenated to the next prompt and passed to the model. It includes unnecessary tokens and summarizing it will maintain the meaning (accuracy) while decreasing the number of tokens. It should be done to the history only and not the new user prompt plus history.

Tokens reduced: input

Expected benefits: Decrease the context length. This allows having more information in the limited context. It also saves on tokens.
Possible disadvantages or risks: Summarization can strip important details and specifics.

3. Controlling the response length and structure

How it works: LLM models allow setting their *"max_tokens"* variable that limits the output length of its response. However, this is not enough as we don't want a truncated response, so it's important to give the model instructions on the output structure.

Tokens reduced: output

Expected benefits: removes unnecessary words and sentences and makes the model focus on just giving the required answer and get the job done.
Possible disadvantages or risks: Aggressive conciseness which might remove some details that are important to know. For example, safety relevant details might be truncated.

4. Semantic response caching

How it works: When a prompt with a similar semantic meaning is asked, the call is not made, and semantic caching returns a stored answer. The previous questions and answers of the users are transformed into vectors and stored in memory with an expiry date.

Tokens reduced: input and output

Expected benefits: Improves latency as we skip the call and directly provide the cached answer.
Possible disadvantages or risks: False hits, where two questions are very close in embedding but require different answers. Another one is an outdated answer after the knowledge base changes, but this can be fixed by revalidating the cache. Personal data might be stored in the cache and given to another user. Overall, this technique seems to have the most disadvantages.

B. Select your preferred approach

I selected technique 1 **top-k filtering** and technique 2 **response length control/structure**.

Why you selected them

They are independent and their implementation does not conflict with the other. They also cover both sides where top-k filtering targets the input tokens while response length targets the output tokens. Among the above techniques, top-k filtering has the most impact on the input tokens as it drastically decreases them – the number of retrieved documents is far less. Same thing applies to response length with output tokens. Moreover, they are relatively easy to implement and provide immediate results. It is important to start with such techniques, then expand to more expensive engineering ones if the trivial ones fail.

How you would implement or evaluate them

Implementation

For top-k filtering, within a retrieval pipeline, the vector database returns a default number of semantic chunks. Instead, it will be set to a hard limit (ex: k=4).

For the response control, first the ***max_tokens*** (variable that controls the maximum number of output tokens) will be set to a specific number (ex: 200). Then, in the **system prompt** of the model, I will give it an instruction to answer within the token limit. If I set the token limit to 200, that is approximately 150 words. So I will add in the system prompt (full prompt in src/config.py):

1. ***Length limit:*** *your entire response must not exceed 150 words.*
2. ***Structure:*** *give the direct answer first, with the most important information first. Then end with exactly one line of the form "Sources: [n], [m]" listing the context passages you used.*
3. ***No filler:*** *no introductions (e.g. "Here is the answer"), no pleasantries, no repetition of the question.*

Evaluation

I will use MTRAG benchmarking dataset. It is an end-to-end human-generated multi-turn RAG benchmark. Ideally, I would run an ablation study, where I have the baseline, top-k only, limited response length, and both combined. Moreover, I will set the temperature to 0 among different ablations to have a fair comparison.

MTRAG has its own evaluation tools – it uses *RB_alg* which uses *BERTSCORE* internally. However, *BERTSCORE* works best with prolonged answers and we are setting max_tokens. Instead, **RAGAS** (a popular evaluation tools for RAG) will be used, it uses an LLM as a judge internally and is less sensitive to response length than BERTSCORE.

What metrics you would use

Number of input and output tokens, RAGAS FactualCorrectness and Faithfulness, monetary cost and latency.

How you would determine whether token reduction negatively affects answer quality

I will compare the baseline and optimized agents on the same tasks using RAGAS FactualCorrectness and Faithfulness. Before running the experiment, I will define an acceptable margin: for example, the optimized agent may lose at most 0.03 in FactualCorrectness, and Faithfulness and should not drop at all. Since scores vary between runs, I report paired bootstrap 95% confidence intervals for every difference instead of running the baseline twice. I will compare scores per task rather than only averages, and manually review the tasks where the optimized agent scored worse, classifying the cause: a gold passage removed by the lower top-k (checked with Recall@k), an answer cut off by the length limit (finish reason length), or missing details due to the shorter answer. Finally, I exported 20 judgments to hand-check against my own assessment (not yet reviewed).

C. Small experimental proposal

My code and documentation is in the Github repo:
https://github.com/wissam6/IMEC-Optimizing-Token-Usage-in-an-AI-Agent

The experimental design is the baseline plus 5 variants. Only the first optimized agent (top-k + response control) was pre-registered; the re-ranking and single-technique variants were follow-ups added after it failed:

Baseline (k=10)
Top-k filtering (k=4) + response control (max_tokens=200), the Combined agent

Top-k filtering (k=4) + re-ranking
Top-k (k=4) + response control (max_tokens=200) + re-ranking
Top-k filtering only (k=4)
Response control only (max_tokens=200)
For the setup, MTRAG Govt domain 30 answerable or partial turns sampled; **gpt-5.4-nano** at temperature 0; retrieval with **bge-small-en-v1.5** embeddings and a FAISS exact inner-product index, querying with MTRAG's rewritten (standalone) questions while the LLM gets the full conversation history; re-ranking with the BAAI/bge-reranker-base cross-encoder (top 20 re-ranked to 4); answers judged by RAGAS with gpt-4.1-mini.

I measured the input and output tokens, the api cost since I was using openai api, LLM latency and retrieval (+ re-ranking) latency, Recall@k that is the share of the gold passages that were retrieved, truncation rate which is how many outputs were cut off by token limit. Faithfulness is the share of answer statements supported by the retrieved passages, and FactualCorrectness F1 is against the reference answer, with precision and recall as diagnostics only, all from one judge pass.

The pass rule, fixed before running, was that FactualCorrectness F1 may drop at most 0.03 and Faithfulness must not drop.

Validation

Smoke test: 5 turns per config read by hand before the full run.

Unit tests: 32 pytest tests (no API calls): retrieval returns exactly k passages, tokens are logged per call, each config changes only its own technique.

Results

The optimized agent (top-k=4 + response-length control) cut total tokens by 52% and cost by 49%, but failed the quality rules (FactualCorrectness F1 −0.048, Faithfulness −0.005). In exploratory follow-ups, **top-k filtering with a cross-encoder reranker, without length control**, cut tokens by 53% and passed both rules (F1 and Faithfulness).

Config	Tokens	Cost / 1k q.	Δ F1	Δ Faithfulness	Rules
Baseline	4,892	$1.12	(0.426)	(0.945)	-
Combined	−52%	$0.57	−0.048 [−0.15, +0.05]	−0.005 [−0.08, +0.07]	fail
Combined + reranker	−52%	$0.56	−0.032 [−0.10, +0.04]	+0.029 [−0.03, +0.10]	fail (F1)
Top-k only	−53%	$0.59	−0.035 [−0.12, +0.05]	−0.023 [−0.08, +0.04]	fail
Top-k + reranker only	−53%	$0.60	−0.008 [−0.07, +0.06]	+0.020 [−0.01, +0.06]	**pass**
Length only	+1%	$1.09	−0.039 [−0.10, +0.02]	+0.002 [−0.06, +0.07]	fail (F1)

Tokens and cost per question vs. baseline (baseline absolute values in brackets). Δ = change vs. baseline with paired bootstrap 95% CI, 30 turns, one run per config.

Lessons learned

1. The tokens should be saved where they are. Most of the token usage coming from the Baseline are input tokens, thus it would have been smart to first run that and based on it choose appropriate technique, instead of choosing a technique first. Limiting the output tokens does not have a direct impact and is not needed even though it is a valid technique.

2. Using top-k filtering is not enough and leads to a drop in quality. This is because with less passages, the agent can lose the only relevant one. However, with a re-ranker, it picks the best candidates with no extra token cost, but with the trade-off of extra latency.

3. The token cap needs a margin. A 200-token cap with a 150-word limit still cut off some answers mid-sentence, so the cap should leave more room above the word limit.

4. 30 turns are too few. Every confidence interval includes zero, so even the passing variant is not a statistically significant result, and a larger sample is needed before choosing a technique.

D. References

Ragas: https://arxiv.org/abs/2309.15217
MTRAG: https://arxiv.org/abs/2501.03468
BAAI/bge-reranker-base: https://huggingface.co/BAAI/bge-reranker-base
https://knowledged.to/notes/ml/top-k-in-rag-search/
https://neuraltrust.ai/blog/ai-token-optimization-guide
https://apxml.com/courses/optimizing-rag-for-production/chapter-5-cost-optimization-production-rag/minimize-llm-token-usage-rag